

CPSC 375 High-Performance Computing

Lecture 4

Message Queues

Linux IPC

- Pipes
- **Message Queues**
- Shared memory
- Semaphores
- Signals
- Sockets

Stream vs. Message

- Pipes are *unstructured* – just a stream of bytes.

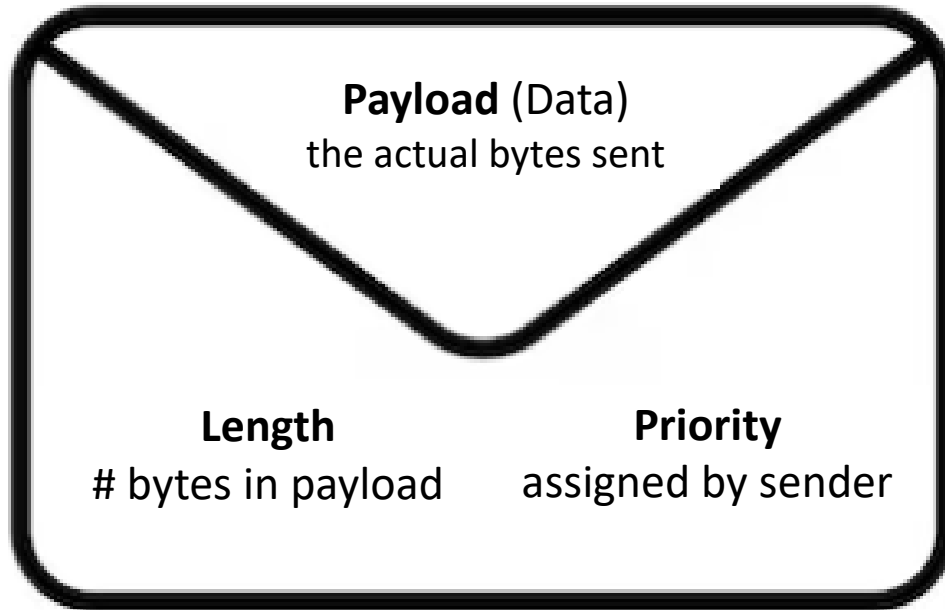


- Message queues are *structured* – discrete records with defined boundaries.
 - e.g., if the sender sends 24 bytes, the receiver gets exactly 24 bytes or an error.

Message Queues

- Kernel-managed message buffers (linked list)
 - kernel preserves the start and end of each message
- Asynchronous: sender and receiver don't need to be active at the same time.
 - sender can place a message in the queue and terminate; the message remains in the kernel until a reader retrieves it.
 - allow processes to exchange messages without blocking
- Messages can be ranked by priority.
 - high-priority messages move to the front of the queue.

What's in the Message?



Queue Attributes

<code>mq_flags</code>	specifies if the queue is in blocking (0) or non-blocking (<code>O_NONBLOCK</code>) mode
<code>mq_maxmsg</code>	max number of messages the queue can hold
<code>mq_msgsize</code>	max size (in bytes) allowed
<code>mq_curmsgs</code>	#messages in the queue waiting to be read

Queue Attributes, cont'd

```
#include <mqueue.h>
```

```
struct mq_attr attr;
```

```
mq_getattr(mqdes, &attr);
```

```
printf("Max messages: %ld\n", attr.mq_maxmsg);
```

```
printf("Current messages waiting: %ld\n",  
      attr.mq_curmsgs);
```

```
printf("Max message size: %ld\n", attr.mq_msgsize);
```

Opening a Message Queue

```
#include <mqueue.h>

mqd_t mq_open(char *name, // must start with /
               int flag,
               mode_t mode, // permission
               struct mq_attr *attr);
```

- **Here, name must begin with /.**
- **Returns a message queue descriptor.**

Sending a Message

```
// default attributes
mqd_t mq = mq_open("/mymq", O_CREAT | O_WRONLY, 0644, NULL);
char *msg = "It's cold!";
mq_send(mq, msg, strlen(msg), 1);

// with attributes with a higher priority
struct mq_attr attr;
attr.mq_maxmsg = 5;
attr.mq_msgsize = 1024;
mqd_t mq = mq_open("/mymq", O_CREAT | O_WRONLY, 0644, &attr);
char *msg = "It's cold!";
mq_send(mq, msg, strlen(msg), 10);
```

Receiving a Message

```
// default attributes
mqd_t mq = mq_open("/mymq", O_RDONLY);
char buf[1024];
int priority;
mq_receive(mq, buf, 1024, &priority);
```

Sending a Message (struct)

```
typedef struct {  
    int id;  
    char name[40];  
    double gpa;  
} Student;
```

```
Student mydata = { .id = 1002,  
                   .name = "Jane Smith",  
                   .gpa = 3.58 };
```

```
mqd_t mq = mq_open("/mymq", O_CREAT | O_WRONLY, 0644, NULL);  
mq_send(mq, (char *)&mydata, sizeof(mydata), 1);
```

Closing and Deleting MQs

```
// Close this process's connection  
mq_close(mq);
```

```
// Delete the queue from the system  
mq_unlink("/mymq");
```

Why Use MQs instead of Pipes?

- **Fault Tolerance**
 - If a receiver process crashes, its assigned tasks remain in the queue for the next available receiver.
- **Zero-Parsing**
 - Unlike Pipes, you don't need to look for delimiters like `\n`. You receive the exact `struct` or buffer size originally sent.
- **Flow Control**
 - You can set a maximum number of messages. If the sender is too fast, the kernel will block it until the receivers catch up, preventing memory exhaustion.
- **Out-of-Order Processing**
 - Priorities ensure that vital messages with aren't stuck behind ordinary messages.

Managing MQs via the Shell

On Linux, Message Qs are visible in a virtual filesystem.

- Mount the queue system:

```
mount -t mqueue none /dev/mqueue
```

- View active queues:

```
ls /dev/mqueue
```

- Inspect a queue:

```
cat /dev/mqueue/mymq
```

Flow Control

- **Blocking (Default)**
 - If the queue is full, the sender blocks. This prevents the producer from overwhelming the system.
- **Non-Blocking:**
 - Opening with `O_NONBLOCK` allows the sender to fail with `EAGAIN` and switch to an alternative task.
- **Asynchronous callback**
 - Use `mq_notify()` to register for an asynchronous callback from the kernel through *signals*.
 - Alternative to blocking (which stalls your process) or polling (which wastes CPU cycles).

Non-Blocking Polling

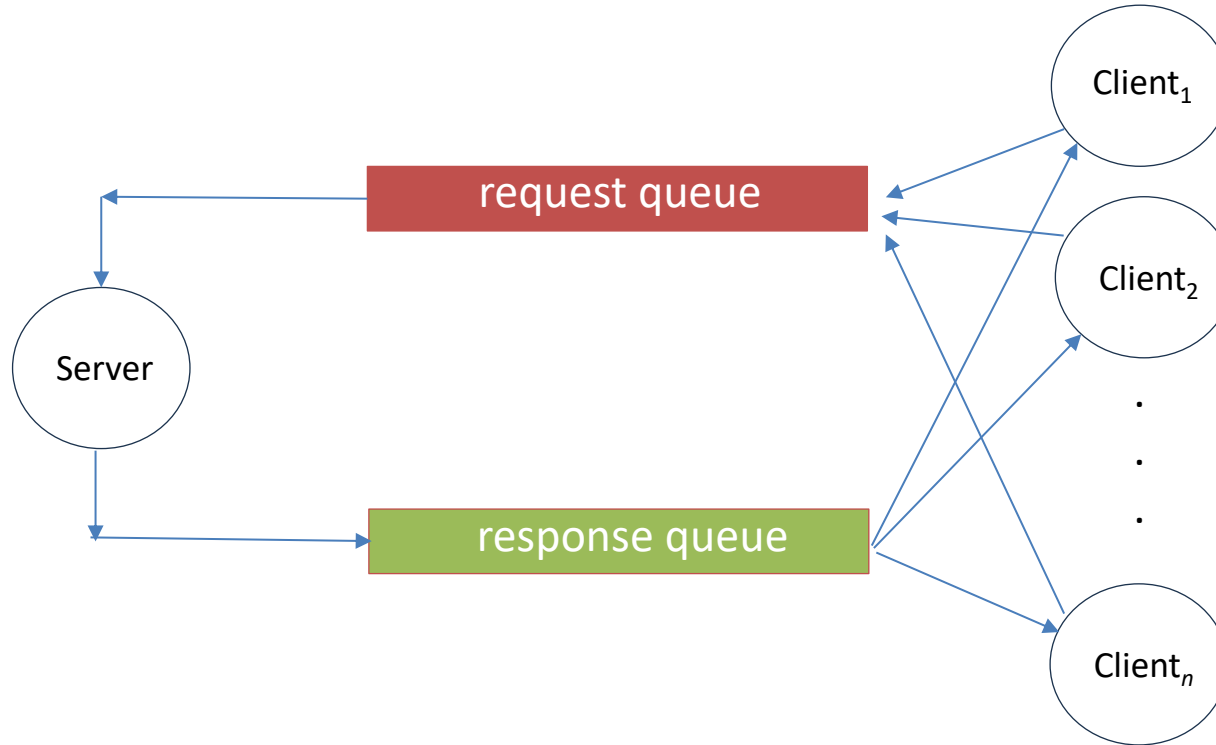
```
// Open in non-blocking mode
mqd_t mq = mq_open("/mymq", O_RDONLY | O_NONBLOCK);
char buf[4096];

while(1) {
    if (mq_receive(mq, buf, 4096, NULL) == -1) {
        if (errno == EAGAIN) {
            // Queue is empty!
            do_useful_stuff();
        }
    }
    . . .
}
```


Flow Control

- **Blocking (Default)**
 - If the queue is full, the sender blocks. This prevents the producer from overwhelming the system.
- **Non-Blocking:**
 - Opening with `O_NONBLOCK` allows the sender to fail with `EAGAIN` and switch to an alternative task.
- **Asynchronous callback**
 - Use `mq_notify()` to register for an asynchronous callback from the kernel through *signals*.
 - Alternative to blocking (which stalls your process) or polling (which wastes CPU cycles).

Example: Client-Server model using MQs



Example: Client-Server Model Using MQs

In a client-server architecture, clients submit requests, and the server processes them and responds.

two separate queues:

- Request queue: clients send data here.
- Response queue: the server sends results back here.

client/server:

- the server creates the queues and listens to the request queue.
- clients open the existing queues to communicate.

